MSK Annotation Suite

Clinical measurement toolkit for knee MRI — TT-TG, Insall–Salvati, Patellar Tilt, and Sulcus Angle with DICOM support and structured CSV export.

license PolyForm-Noncommercial-1.0.0

Table of Contents

- MSK Annotation Suite
  - Table of Contents
  - Running the Code
  - Deploying to Render
  - What We Export & Why
    - The Golden Thread: How Coordinates Stay Reproducible
    - CSV Export Architecture
      - 1. Session Annotation CSV (`src/app/lib/sessionAnnotationCsv.ts`)
        - Core Columns (12 columns)
        - Extended ML Columns (10+ columns)
        - `measurementType` Values and Their Geometry
      - 2. Protocol Measurement CSV (`src/app/lib/protocolMeasurementCsv.ts`)
        - How Annotations Are Matched to Protocol Steps
        - Cross-Protocol Annotation Borrowing
        - Record Types
        - Full Column Reference
        - Example: A Complete TT-TG Protocol Row
    - Measurement Protocols & Required Annotations
      - Per-Protocol Landmark Overlap
      - What This Means for ML
    - DICOM Spatial Metadata
  - DICOM Data Architecture
    - Volume Structure
    - Coordinate Systems
    - Reference Line System
  - Machine Learning: Predicting Measurements from DICOM
    - Problem Formulation
    - Input → Output Mapping
      - Protocol 1: TT-TG (Axial)
      - Protocol 2: Insall–Salvati (Sagittal)
      - Protocol 3: Patellar Tilt (Axial)
      - Protocol 4: Sulcus Angle (Axial)
      - Protocol 5: Sulcus Angle 3 cm (Cross-Plane)
      - Protocol 6: Caton–Deschamps (Sagittal)
    - Recommended Architecture: Hybrid 2.5D Landmark Heatmap + Regression
      - Key Design Decisions
      - PyTorch Pseudocode (Core Model)
    - Architecture Option B: Direct ViT Regression (with DINOv2)
    - Architecture Option C: 3D CNN with Attention
    - Data Pipeline Sketch
      - Data Preprocessing Details
    - Loss Functions
    - Training Strategy
      - Framework: MONAI
      - Pretraining: DINOv2 (not SimCLR/BYOL)
      - Curriculum Learning (Protocol Difficulty Ordering)
      - Full Training Loop
    - Evaluation Metrics
    - Dataset Requirements
    - Clinical Gaps & Production Considerations
      - Annotation Versioning
      - Two-Stage Inference for Sulcus Angle 3 cm
      - MONAI Integration
    - Next Steps
  - License

Running the Code

```
pnpm install
pnpm dev
```

> **Note:** This project uses **pnpm**. If you don't have it, install via `npm install -g pnpm` .

Deploying to Render

This project is configured for one-click deployment on Render as a static site. The `render.yaml` will be auto-detected.

1. Push this repo to GitHub/GitLab
2. On Render, create a new **Static Site**
3. Connect your repository
4. Render will auto-detect the `render.yaml` config, or set manually:
   - **Build Command:** `pnpm install && pnpm build`
   - **Publish Directory:** `dist`

What We Export & Why

The tool produces **two complementary CSV formats** that together form a complete ground-truth dataset for ML training. They serve different purposes:

CSV	Purpose	Granularity	Best for
Session Annotation CSV	Raw drawing stream	One row per annotation (line, point, angle)	Landmark detection training
Protocol Measurement CSV	Computed clinical results	One row per measurement, grouped by protocol run	Value regression training + clinical audit

The Golden Thread: How Coordinates Stay Reproducible

The fundamental challenge is that browser viewport sizes vary, but the DICOM image pixels and patient anatomy do not. Every annotation stores an **`imageScale`** object atomically with the annotation:

```
imageScale: {
  x: imgWidth / drawWidth,             // CSS→image pixel scale factor
  y: imgHeight / drawHeight,
  offsetX: (viewportW - drawW) / 2,    // CSS offset from viewport edge
  offsetY: (viewportH - drawH) / 2,
}
```

This is captured **at the moment the annotation is drawn** — before the user ever resizes their window. Combined with DICOM pixel spacing from the volume header, coordinates are always recoverable.

CSS px → Patient mm in practice (for a 256×256 image displayed at 400×400 px with 0.35 mm/pixel spacing):

```
Annotation drawn at CSS (245, 312)
  → subtract offsetX/offsetY to get canvas-relative position
  → multiply by imageScale.x / imageScale.y to get image-pixel coordinates
  → multiply by DICOM pixel spacing to get physical mm (x,y)
  → multiply by affine transform to get 3D patient space (x,y,z)
  → z-axis computed as sliceIndex × sliceSpacing for through-plane position
```

Critically, the offset subtraction cancels out for *distances* ($dx = x_1 - x_0$, so offset cancels), but is essential for *point* and *perpendicular* measurements where absolute position matters.

CSV Export Architecture

1. Session Annotation CSV (`src/app/lib/sessionAnnotationCsv.ts`)

Every click, every line, every measurement — raw and unprocessed. Each row is exactly one drawing action by the clinician. This is the most granular training target for landmark prediction models.

Core Columns (12 columns)

Column	Type	Example	Meaning
`annotationId`	UUID	`a1b2c3d4-...`	Globally unique per drawing
`patientId`	string	`MRN12345`	DICOM PatientID tag `(0010,0020)`
`laterality`	enum	`left`	Which knee (`left` / `right`)
`sequenceName`	string	`Ax T2 FSE`	DICOM Series Description — identifies the pulse sequence
`plane`	enum	`axial`	Anatomical plane of the volume
`measurementType`	enum	`perpendicular`	Drawing tool used: `line` , `distance` , `point` , `angle` , `perpendicular`
`value`	string	`12.4 mm`	In-viewport measurement label (CSS-pixel derived, approximate)
`units`	string	`mm`	Unit suffix parsed from the value string
`sliceIndex`	int	`17`	0-based slice within the volume where annotation lives
`annotatedBy`	string	`Dr. Smith`	Clinician name
`annotatorEmail`	string	`smith@hospital.org`	Clinician email for audit trail
`timestamp`	ISO-8601	`2025-03-15T14:22:31.000Z`	Exact capture time

Extended ML Columns (10+ columns)

Column	Type	Meaning
`label`	string	Human-readable label, e.g. `"Posterior femoral condyle line"`
`workflowStepId`	string	Which protocol step this fulfills (`condyle-line` , `trochlear-groove` , etc.)
`groupId`	string	Links annotations of the same protocol run (e.g. `tt-tg-20250315-001`)
`propagateAcrossSlices`	bool	If true, annotation overlay is visible on all slices in the plane
`points_count`	int	Number of points in this annotation's geometry
`p0_x` … `p3_y`	float	Up to 4 flattened CSS-pixel points (8 columns)
`baseline_p0_x` … `baseline_p1_y`	float	Baseline/reference line endpoints (4 columns), resolved via `baseLineId`
`points_json`	JSON	Full `[{x, y}, ...]` point array for programmatic consumption
`baseline_points_json`	JSON	Full baseline point array

> **Why two representations?** The flattened `p0_x` … `p3_y` columns let you filter/sort in Excel. The `points_json` column gives you the full array for Python/ML pipelines without manual column splitting.

`measurementType` Values and Their Geometry

Type	Point Count	Typical Use	Example
`point`	1	Joint line placement, calibration	Femorotibial joint point
`line`	2	Condyle line, patella axis	Posterior femoral condyle line
`distance`	2	Length measurements	Patella length (LP), tendon length (LT)
`angle`	3	Angle between two lines (vertex at pt[1])	Any 3-point angle measurement
`perpendicular`	2	Branch off a reference line	TT-TG groove/tubercle offsets — pt[0]=anchor on ref line, pt[1]=landmark

What makes this CSV valuable for ML: Every row is a self-contained training example. The `p0_x` … `p3_y` columns are the regression targets for a landmark detector. The `label` and `workflowStepId` columns are the classification labels. The `imageScale` (embedded in `points_json` via the row's capture context) lets you convert to image-pixel coordinates. The `sliceIndex` tells you which slice to feed into a 2.5D model.

---

2. Protocol Measurement CSV (`src/app/lib/protocolMeasurementCsv.ts`)

Structured, computed, clinically meaningful. This CSV groups annotations by protocol run, infers which annotations fulfill which steps, runs the protocol's `compute()` function, and exports the final clinical values alongside all intermediate data — in **physical mm**, not CSS pixels.

How Annotations Are Matched to Protocol Steps

The exporter uses a **3-pass inference system** to determine which annotations belong to which protocol steps:

1. **Pass 1 — Exact `workflowStepId` match:** Most reliable. Annotations created inside the Measurement Workflow UI have `workflowStepId` set to the active step's ID (`condyle-line` , `trochlear-groove` , etc.). These are claimed first.
2. **Pass 2 — Label + primitive match:** Fallback for re-imported or manually-labeled annotations. If an annotation's `label` matches a step's `label` and the `measurementType` is compatible with the step's `primitive` , it's claimed.
3. **Pass 3 — Primitive type match in step order:** Last resort. Unclaimed annotations are matched to unfulfilled steps by primitive type, iterating steps in declaration order so two same-primitive steps (e.g. both `line` steps in Patellar Tilt) don't steal each other's measurement.

> **Deduplication:** A `usedIds` set prevents any annotation from being claimed by two different steps within the same protocol.

Cross-Protocol Annotation Borrowing

A critical feature: **annotations drawn for one protocol can satisfy steps in another.** For example, the posterior femoral condyle line drawn during TT-TG is automatically reused by Patellar Tilt (both need it). The exporter:

- Groups measurements by `patientId + groupId` composite key
- For each group, checks if any steps are missing
- Searches across ALL measurements (same patient, same plane) for compatible candidates
- Shallow-clones borrowed annotations into the borrowing group (the original stays in its own group)

This means a clinician can run 3 protocols on the same knee and only draw the condyle line **once**.

Record Types

Each row has a `recordType` field that indicates its role:

`recordType`	Meaning	Key Columns Populated
`protocol_result`	Final clinical computation for one protocol run	`resultValue` , `resultUnit` , `resultSummary` , `interpretation` , `stepMeasurementIdsJson`
`step_measurement`	Individual annotation within a protocol run	`measurementId` , `stepId` , `p0_x…p3_z` (in mm), `stepPointsMmJson`
`raw_measurement`	Annotation that didn't belong to any recognized protocol	All geometry columns, no protocol metadata

Full Column Reference

Column	Source	Description
`recordType`	generated	`protocol_result` , `step_measurement` , or `raw_measurement`
`patientId`	DICOM / annotation	Patient identifier
`sessionUser`	session	Clinician name (at export time)
`sessionUserEmail`	session	Clinician email (at export time)
`laterality`	annotation	`left` or `right`
`sequenceName`	DICOM	Pulse sequence description
`plane`	annotation	Anatomical plane
`protocolId`	computed	`tt-tg` , `insall-salvati` , `patellar-tilt` , `sulcus-angle` , `sulcus-angle-3cm` , `caton-deschamps`
`protocolLabel`	protocol def	Human-readable name
`groupId`	annotation	Groups annotations of the same protocol run
`measurementId`	annotation	UUID of the individual annotation
`stepId`	annotation	Protocol step ID (`condyle-line` , `is-patella-length` , etc.)
`stepLabel`	protocol def	Human-readable step name
`primitive`	protocol def	Expected primitive: `line` , `distance` , `angle` , `point`
`measurementType`	annotation	Actual drawing type (`line` , `distance` , `point` , `angle` , `perpendicular`)
`sliceIndex`	annotation	0-based slice number
`pointCount`	annotation	Number of points in this annotation
`resultValue`	computed	Final clinical measurement value (number as string)
`resultUnit`	computed	`mm` , `°` , or empty (for ratios)
`resultSummary`	computed	Human-readable, e.g. `"TT-TG = 12.4 mm"`
`interpretation`	computed	Clinical interpretation, e.g. `"Elevated (>20 mm) — associated with patellar instability."`
`measurementLabel`	annotation	Human-readable label
`measurementValue`	computed	Per-annotation computed value (distance in mm or angle in °)
`measurementUnit`	computed	Per-annotation unit
`p0_x` … `p3_z`	computed	Up to 4 points × 3 coordinates = 12 columns. **Coordinates are in physical mm**: `x_mm = (cssX - offsetX) × imageScale.x × pixelSpacing.x` , `y_mm = (cssY - offsetY) × imageScale.y × pixelSpacing.y` , `z_mm = sliceIndex × sliceSpacing` . Empty string for missing points.
`stepPointsMmJson`	computed	JSON array of `[{x, y, z?}, ...]` — the annotation points converted to physical mm
`baselineId`	annotation	UUID of the baseline annotation (if this is a perpendicular measurement)
`baseline_p0_x` … `baseline_p1_z`	computed	Baseline endpoints in physical mm (6 columns)
`baselinePointsMmJson`	computed	JSON array of baseline points in physical mm
`pointsJson`	duplicated	Same as `stepPointsMmJson` (legacy compatibility)
`baselinePointsJson`	duplicated	Same as `baselinePointsMmJson` (legacy compatibility)
`stepMeasurementIdsJson`	computed	JSON array of all measurement IDs used in this protocol run

Example: A Complete TT-TG Protocol Row

Here's what `protocol_result` and `step_measurement` rows look like for a TT-TG measurement. The condyle line was drawn, then two perpendicular branches for groove and tubercle:

```
recordType,patientId,sessionUser,...,protocolId,protocolLabel,groupId,measurementId,stepId,stepLabel,p
protocol_result,MRN001,Dr. Smith,...,tt-tg,TT-TG,tt-tg-001,,,,,,,,,12.4,mm,TT-TG = 12.4 mm,Within norm
step_measurement,MRN001,Dr. Smith,...,tt-tg,TT-TG,tt-tg-001,<uuid1>,condyle-line,Posterior femoral con
step_measurement,MRN001,Dr. Smith,...,tt-tg,TT-TG,tt-tg-001,<uuid2>,trochlear-groove,Deepest point of
step_measurement,MRN001,Dr. Smith,...,tt-tg,TT-TG,tt-tg-001,<uuid3>,tibial-tubercle,Most anterior poin
```

> **Key insight:** The `resultValue` (12.4) is repeated on every row within the group. This makes it easy to filter for unique protocol results with `recordType = protocol_result` , or to join step-level geometry back to clinical outcomes with `groupId` .

Measurement Protocols & Required Annotations

The suite implements **6 knee MRI measurement protocols**. Each protocol defines a sequence of annotation steps, a required plane, and a `compute` function that derives the clinical value from annotated landmarks.

Protocol	ID	Plane	Input Annotations	Output
TT-TG	`tt-tg`	Axial	① Posterior condyle line (2 pts, `line`) ② Trochlear groove point (1 pt, `point` — perpendicular branch) ③ Tibial tubercle point (1 pt, `point` — perpendicular branch)	Distance (mm) between groove & tubercle, projected along condyle line. Normal <15 mm, borderline 15–20 mm, elevated >20 mm.
Insall–Salvati	`insall-salvati`	Sagittal	① Patella length LP (2 pts, `distance` — longest patellar axis including non-articular portions) ② Patellar tendon length LT (2 pts, `distance` — inferior pole to tibial tuberosity)	Ratio LT/LP. Normal 0.8–1.2, patella alta >1.2, patella baja <0.8.
Patellar Tilt	`patellar-tilt`	Axial	① Posterior condyle line (2 pts, `line`) ② Patella transverse axis (2 pts, `line` — widest patellar width)	Acute angle (°). Normal <10°, borderline 10–20°, abnormal >20°.
Sulcus Angle	`sulcus-angle`	Axial	① Medial condyle→sulcus line (2 pts, `line`) ② Lateral condyle→sulcus line (2 pts, `line` — both share groove as endpoint)	Obtuse angle (°) of trochlear sulcus, supplement of the acute angle if needed. Normal <145°, dysplasia >145°.
Sulcus Angle (3 cm)	`sulcus-angle-3cm`	Sagittal → Axial	① Joint line point on sagittal (1 pt, `point`) — generates 30 mm reference line ② Medial→sulcus line on axial ③ Lateral→sulcus line on axial	Sulcus angle at 3 cm superior to femorotibial joint line. Normal <145°, dysplasia >145°.
Caton–Deschamps	`caton-deschamps`	Sagittal	① Patellar articular surface A (2 pts, `distance` — cartilage-bearing portion only, superior→inferior margin) ② Patellar height B (2 pts, `distance` — inferior articular margin → anterior tibial margin)	Ratio B/A. Normal 0.6–1.3, patella alta >1.3, patella baja <0.6. Less affected by knee flexion than Insall–Salvati.

Per-Protocol Landmark Overlap

Many landmarks repeat across protocols. The exporter's **borrowing system** (described above) leverages this so clinicians don't redraw the same anatomy:

```
Posterior femoral condyle line  ← shared by TT-TG, Patellar Tilt, Sulcus Angle
Trochlear groove (sulcus)       ← shared by Sulcus Angle, Sulcus Angle 3cm
Medial/lateral condyle peaks    ← shared by Sulcus Angle, Sulcus Angle 3cm
Patella landmarks               ← unique to Insall–Salvati and Caton–Deschamps
```

Total unique landmarks across all 6 protocols: ~12–15 distinct 2D points.

What This Means for ML

For each protocol, the model must either:

- **(A) Predict the raw landmark coordinates** on the correct slice, then compute the clinical value using the same `compute` functions (preferred — interpretable, auditable, and the exported functions are directly reusable)
- **(B) Predict the clinical value directly** (end-to-end, simpler training target but black-box and hard to debug)

The export CSVs provide ground truth for both approaches:

- **Session CSV** → `points_json` / `p0_x…p3_y` → raw CSS-pixel landmarks → train a landmark heatmap detector
- **Protocol CSV** → `stepPointsMmJson` → landmarks in physical mm → train or validate with physical coordinates
- **Protocol CSV** → `resultValue` → final clinical measurement → train a direct regression model

DICOM Spatial Metadata

Every DICOM volume carries spatial registration tags needed to convert between pixel, voxel, and patient coordinates:

Tag	Name	Purpose
`(0028,0030)`	Pixel Spacing	`[rowSpacing, colSpacing]` in mm — converts image pixels → physical mm
`(0020,0037)`	Image Orientation Patient	6 direction cosines `[rowX,rowY,rowZ, colX,colY,colZ]` — defines slice plane orientation in patient space
`(0020,0032)`	Image Position Patient	`[x,y,z]` of first voxel of first slice in patient mm
`(0018,0050)`	Slice Thickness	Nominal slice gap in mm

These are stored in the volume header as `pixDims` , `imageOrientationPatient` , `imagePositionPatient` , `sliceDirection` , and `sliceSpacing` . The `dicomAffine.ts` module uses them to build the **3D affine transform** that maps between voxel space `(row, col, slice)` and patient space `(x, y, z)` — enabling cross-plane landmark mapping (e.g., sagittal point → axial slice navigation for the 3 cm sulcus angle).

DICOM Data Architecture

Volume Structure

DICOM series are loaded independently and organized by **laterality** (left/right knee) and **plane** (sagittal/coronal/axial). The top-level study structure is:

```
DicomStudy {
  studyName: string
  patientId: string
  patientName: string
  studyInstanceUID: string
  knees: {
    left:  { volumes: Partial<Record<Plane, DicomVolume>> }
    right: { volumes: Partial<Record<Plane, DicomVolume>> }
  }
}
```

Within each knee, a `DicomVolume` is a complete 3D stack:

```
DicomVolume {
  imageData: Uint8Array          // [slice][row][col], 8-bit windowed
  header: {
    dims:    [3, cols, rows, slices]   // NIfTI convention
    pixDims: [1, dx_col, dy_row, dz_slice]  // mm/pixel
    datatypeCode: 2                   // uint8
    scl_slope: 1, scl_inter: 0
    // DICOM spatial registration:
    imageOrientationPatient?: [r0,r1,r2, c0,c1,c2]
    imagePositionPatient?:    [x, y, z]
    sliceDirection?:          [sx, sy, sz]
    sliceSpacing?:            number
  }
  dataRange: { min: number; max: number }
  defaultWindowLevel: { window: number; level: number }
  sliceCount: number
  plane: 'sagittal' | 'coronal' | 'axial'
  laterality: 'left' | 'right'
  seriesDescription: string          // DICOM Series Description
  patientId: string                  // DICOM PatientID
  patientName: string
  studyInstanceUID: string
  seriesInstanceUID: string
  origin: 'dicom'
}
```

Key dimensions per plane (from a typical knee MRI):

Plane	dims[1] (cols)	dims[2] (rows)	dims[3] (slices)	pixDims[1] (dx)	pixDims[2] (dy)	pixDims[3] (dz)
Axial	~256–512	~256–512	~20–40	~0.3–0.5 mm	~0.3–0.5 mm	~3.0–5.0 mm
Sagittal	~256–512	~256–512	~20–40	~0.3–0.5 mm	~0.3–0.5 mm	~3.0–5.0 mm
Coronal	~256–512	~256–512	~20–70	~0.3–0.5 mm	~0.3–0.5 mm	~0.5–3.0 mm

Coordinate Systems

There are **three coordinate spaces** to track:

Space	Description	Example
CSS pixels	Browser viewport overlay coordinates. Annotation points are stored here	`{x: 245.3, y: 312.7}`
Image pixels	Native DICOM pixel grid. Converted via `imageScale`	`{x: 156.2, y: 198.9}`
Patient mm	Physical 3D patient space. Converted via pixel spacing + affine	`{x: -45.2, y: 12.7, z: 89.3}`

Conversion chain:

```
CSS px  →  imageScale  →  Image px  →  pixDims  →  Patient mm
                                                    →  affine  →  3D Patient (x,y,z)
```

The `imageScale` is stored atomically with each annotation so coordinates can always be recovered:

```
imageScale: {
  x: imgWidth / drawWidth,     // CSS→image scale factor
  y: imgHeight / drawHeight,
  offsetX: (viewportW - drawW) / 2,  // CSS offset of drawn image
  offsetY: (viewportH - drawH) / 2,
}
```

Reference Line System

The **Sulcus Angle (3 cm)** protocol uses a cross-plane reference line:

1. Clinician places a **joint line point** on a sagittal slice
2. A **reference line** appears 3 cm (30 mm) *superior* to that point
3. Clicking the reference line navigates the axial viewer to that level
4. Clinician draws sulcus angle lines on the axial slice at 3 cm above the joint

ML implication: For the 3 cm sulcus angle, the model must either:

- Learn to identify the femorotibial joint line on sagittal slices, compute the 3 cm offset in volume space, and select the correct axial slice
- Or be given the axial slice index as an auxiliary input (simpler, but requires the sagittal→axial slice mapping)

Machine Learning: Predicting Measurements from DICOM

> **Note:** This section describes the **planned ML architecture** for training models on the annotation data exported by this tool. None of the PyTorch code shown here is part of the running application — it is provided as a reference for researchers building ML pipelines on top of the exported CSVs.

Problem Formulation

Given: Three co-registered 3D DICOM volumes — sagittal V_S, coronal V_C, axial V_A — each of shape (S, H, W) single-channel, with known pixel spacing (dx, dy, dz).

Predict: One or more clinical measurements. There are two output granularities:

Level	Output	Pros	Cons
Landmark-level	2D/3D coordinates of anatomical keypoints (condyle endpoints, groove point, patella poles, etc.) on identified slices	Interpretable, auditable, can compute any protocol from landmarks	Requires per-landmark annotation
Value-level	Direct clinical values (distances in mm, angles in °, ratios)	Simpler training target	Black-box, hard to debug, protocol-coupled

Recommendation: Landmark-level prediction — then compute measurements using the existing protocol `compute` functions. This mirrors how clinicians work and produces auditable outputs.

Input → Output Mapping

For each measurement protocol, here is the exact landmark set the model must predict:

Protocol 1: TT-TG (Axial)

```
Input:  Axial volume V_A
Output: On the correct axial slice s*:
  L1 = (x1, y1) — posterior condyle line endpoint 1
  L2 = (x2, y2) — posterior condyle line endpoint 2
  G  = (xg, yg) — deepest trochlear groove point
  T  = (xt, yt) — most anterior tibial tubercle point
→ TT-TG distance = |projectAlongCondyle(T) - projectAlongCondyle(G)| × condyleLineLength
```

Protocol 2: Insall–Salvati (Sagittal)

```
Input:  Sagittal volume V_S
Output: On the correct sagittal slice s*:
  P1 = (x1, y1) — superior patella pole
  P2 = (x2, y2) — inferior patella pole (LP)
  T1 = (x3, y3) — inferior patellar pole (tendon origin)
  T2 = (x4, y4) — tibial tuberosity insertion (LT)
→ Insall–Salvati ratio = ||T1−T2|| / ||P1−P2||
```

Protocol 3: Patellar Tilt (Axial)

```
Input:  Axial volume V_A
Output: On the correct axial slice s*:
  C1, C2 — posterior femoral condyle line (2 points)
  P1, P2 — patella transverse axis (2 points)
→ Patellar Tilt = acuteAngleBetween(line(C1,C2), line(P1,P2))
```

Protocol 4: Sulcus Angle (Axial)

```
Input:  Axial volume V_A
Output: On the correct axial slice s*:
  M — medial condyle peak
  S — deepest trochlear groove (sulcus)
  L — lateral condyle peak
→ Sulcus Angle = angleBetween(vectors(S→M, S→L)), supplement if < 90°
```

Protocol 5: Sulcus Angle 3 cm (Cross-Plane)

```
Input:  Sagittal volume V_S + Axial volume V_A
Step A (Sagittal):
  J = (x_j, y_j) — femorotibial joint line point on slice s_sag
  Compute axial slice s* = sliceAtOffsetAbove(s_sag, 30mm, V_S.affine, V_A.affine)
Step B (Axial, at slice s*):
  M, S, L — same as Sulcus Angle
→ Sulcus Angle (3 cm)
```

Protocol 6: Caton–Deschamps (Sagittal)

```
Input:  Sagittal volume V_S
Output: On the correct sagittal slice s*:
  A1, A2 — patellar articular surface (superior→inferior margin)
  B1, B2 — inferior articular margin → anterior tibial margin
→ CDI = ||B1−B2|| / ||A1−A2||
```

Total landmark set: ~12–15 unique 2D points across all 6 protocols. Many landmarks repeat across protocols (e.g., the posterior condyle line is the same for TT-TG, Patellar Tilt, and Sulcus Angle).

Recommended Architecture: Hybrid 2.5D Landmark Heatmap + Regression

This is the **strongest baseline** — clinically validated in similar work (e.g., total knee arthroplasty planning, hip landmark detection).

```
┌──────────────────────────────────────────────────────────┐
│                    INPUT: 3 Volumes                       │
│  V_S (Sagittal)   V_C (Coronal)   V_A (Axial)            │
│  [S_s, H, W, 1]   [S_c, H, W, 1]  [S_a, H, W, 1]         │
└──────┬──────────────────┬──────────────────┬─────────────┘
       │                  │                  │
       ▼                  ▼                  ▼
┌─────────────┐   ┌─────────────┐   ┌─────────────┐
│ ResNet-50   │   │ ResNet-50   │   │ ResNet-50   │
│ (2D per-slice│  │ (2D per-slice│  │ (2D per-slice│
│  encoder)   │   │  encoder)   │   │  encoder)   │
│ → f_S[s]    │   │ → f_C[s]    │   │ → f_A[s]    │
│  [S_s, h, w, │  │  [S_c, h, w, │  │  [S_a, h, w, │
│   C]        │   │   C]        │   │   C]        │
└──────┬──────┘   └──────┬──────┘   └──────┬──────┘
       │                  │                  │
       ▼                  ▼                  ▼
┌──────────────────────────────────────────────────────────┐
│                Slice-Aware Feature Aggregation            │
│  For each volume: 3D Conv (1×3×3) or LSTM along slice axis │
│  + Cross-plane attention between sagittal↔axial↔coronal   │
│  → f_fused[s_a, s_s, s_c]  shared 3D feature volume       │
└────────────────────────┬─────────────────────────────────┘
                         │
                         ▼
┌──────────────────────────────────────────────────────────┐
│                  Shared Decoder (U-Net style)             │
│  For each target plane/slice: upsample → heatmap per landmark │
│  Output: K heatmaps of shape [H, W] per relevant slice    │
│  Each heatmap = Gaussian centered at landmark location    │
└────────────────────────┬─────────────────────────────────┘
                         │
                         ▼
┌──────────────────────────────────────────────────────────┐
│             Landmark Extraction + Measurement Compute     │
│  Soft-argmax → (x̂, ŷ) per landmark                        │
│  Apply protocol compute() → TT-TG, Insall-Salvati, etc.   │
└──────────────────────────────────────────────────────────┘
```

Key Design Decisions

1. **2.5D over full 3D:** Per-slice 2D CNN encoders are more parameter-efficient than 3D convs (which are k^3 vs k^2). Knee MRI slices have high in-plane resolution (~0.35 mm) but thick slice spacing (~3 mm) — 2D features are richer than inter-slice features.
2. **HRNet or ViT-based backbone (not ResNet-50):** HRNet maintains high-resolution representations throughout the network without a decoder bottleneck — this is specifically why it outperforms ResNet-based approaches on medical landmark detection. Alternatively, **MedSAM** (SAM fine-tuned on medical images, 2024) provides a ViT-H encoder pre-trained on medical imaging that transfers exceptionally well to MRI. A third option: use **DINOv2** (`dinov2_vitb14`) weights directly — they transfer better than ImageNet-supervised weights for MRI because they learn patch-level features that capture fine anatomical texture.
3. **Slice-aware aggregation:** A lightweight 3D conv (1×3×3 kernel) or bidirectional LSTM along the slice dimension fuses context from neighboring slices. This helps identify the *correct* slice for each measurement.
4. **Cross-plane attention (fully implemented below):** A transformer block that attends sagittal features ↔ axial features ↔ coronal features enables the model to learn cross-plane anatomical relationships (critical for Sulcus Angle 3cm). The `CrossPlaneAttention` module pools each plane's slice features via GAP, concatenates them as tokens, and applies multi-head self-attention with residual layer-norm.
5. **Heatmap output (not direct regression):** Gaussian heatmaps are the gold standard for landmark detection (used in Hourglass, HRNet, etc.). They are spatially interpretable and produce well-calibrated uncertainty.

PyTorch Pseudocode (Core Model)

```python
import math
import torch
import torch.nn as nn
import torch.nn.functional as F


# ═══════════════════════════════════════════════════════════════════════
# Cross-Plane Attention Module
# ═══════════════════════════════════════════════════════════════════════

class CrossPlaneAttention(nn.Module):
    """
    Fuses features from sagittal, coronal, and axial planes via multi-head
    self-attention.  Each plane's slice features are pooled (GAP) into a
    single token per slice, then the full set of [S_s + S_c + S_a] tokens
    attends across planes.  Residual + LayerNorm preserves per-plane identity.
    """
    def __init__(self, channels: int, num_heads: int = 8):
        super().__init__()
        self.attn = nn.MultiheadAttention(channels, num_heads, batch_first=True)
        self.norm = nn.LayerNorm(channels)

    def forward(self, f_sag, f_cor, f_ax):
        # f_sag: [B, S_s, C, h, w],  f_cor: [B, S_c, C, h, w], etc.
        # GAP over spatial dims → one token per slice
        t_s = f_sag.mean(dim=(3, 4))   # [B, S_s, C]
        t_c = f_cor.mean(dim=(3, 4))   # [B, S_c, C]
        t_a = f_ax.mean(dim=(3, 4))    # [B, S_a, C]

        tokens = torch.cat([t_s, t_c, t_a], dim=1)  # [B, S_s+S_c+S_a, C]
        out, _ = self.attn(tokens, tokens, tokens)
        out = self.norm(out + tokens)                # residual

        # Split back into per-plane tokens
        S_s, S_c = t_s.shape[1], t_c.shape[1]
        return {
            'sagittal': out[:, :S_s],
            'coronal':  out[:, S_s:S_s + S_c],
            'axial':    out[:, S_s + S_c:],
        }


# ═══════════════════════════════════════════════════════════════════════
# Wing Loss — superior to L1/MSE for landmark coordinate regression
# ═══════════════════════════════════════════════════════════════════════

class WingLoss(nn.Module):
    """
    Gives higher gradient weight to small errors (< w) and linear behavior
    for large errors.  The constant C = w − w·ln(1 + w/ε) ensures continuity
    at the transition point |x| = w.

    Reference: Feng et al., "Wing Loss for Robust Facial Landmark Localisation
               with Convolutional Neural Networks", CVPR 2018.
    """
    def __init__(self, w: float = 10.0, epsilon: float = 2.0):
        super().__init__()
        self.w = w
        self.epsilon = epsilon
        self.C = w - w * math.log(1 + w / epsilon)

    def forward(self, pred: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
        diff = (pred - target).abs()
        loss = torch.where(
            diff < self.w,
            self.w * torch.log(1 + diff / self.epsilon),
            diff - self.C,
        )
        return loss.mean()


# ═══════════════════════════════════════════════════════════════════════
# Core Model: 2.5D Multi-Plane Landmark Detector
# ═══════════════════════════════════════════════════════════════════════

class KneeLandmarkModel(nn.Module):
    """
    2.5D multi-plane landmark detection model with HRNet / ViT backbone,
    cross-plane attention, and per-protocol slice classifiers.

    Input:  [B, S_s, 1, H, W]  sagittal volume
            [B, S_c, 1, H, W]  coronal volume
            [B, S_a, 1, H, W]  axial volume

    Output:
        heatmaps:   Dict[str, Tensor]  — per-protocol heatmap stack [B, K_p, H, W]
        coords:     Dict[str, Tensor]  — soft-argmax coordinates  [B, K_p, 2]
        slice_logits: Dict[str, Tensor] — per-protocol slice predictions [B, max_slices]
        measurements: Dict[str, Tensor] — computed clinical values [B, 1..3]
    """

    # ── protocol definitions ──────────────────────────────────────────
    PROTOCOLS = {
        'tt_tg': {
            'plane': 'axial',
            'landmarks': ['condyle_p1', 'condyle_p2', 'trochlear_groove', 'tibial_tubercle'],
            'output_values': 1,  # distance mm
        },
        'insall_salvati': {
            'plane': 'sagittal',
            'landmarks': ['patella_sup', 'patella_inf', 'tendon_origin', 'tendon_insertion'],
            'output_values': 1,  # ratio
        },
        'patellar_tilt': {
            'plane': 'axial',
            'landmarks': ['condyle_p1', 'condyle_p2', 'patella_p1', 'patella_p2'],
            'output_values': 1,  # angle °
        },
        'sulcus_angle': {
            'plane': 'axial',
            'landmarks': ['medial_condyle', 'sulcus_deepest', 'lateral_condyle'],
            'output_values': 1,  # angle °
        },
        'sulcus_angle_3cm': {
            'plane': 'axial',           # axial landmarks; sagittal joint-line inferred
            'landmarks': ['joint_line_sag', 'medial_condyle', 'sulcus_deepest', 'lateral_condyle'],
            'output_values': 1,  # angle °
        },
        'caton_deschamps': {
            'plane': 'sagittal',
            'landmarks': ['patella_sup_articular', 'patella_inf_articular',
                          'inf_articular_margin', 'ant_tibial_margin'],
            'output_values': 1,  # ratio
        },
    }

    def __init__(self, num_landmarks: int = 15, backbone: str = 'hrnet_w32',
                 max_slices: int = 80):
        super().__init__()
        self.max_slices = max_slices

        # ── 2D Encoder (HRNet or ViT-based) ──────────────────────────
        # HRNet maintains high-res features; no decoder bottleneck needed.
        # To use HRNet: pip install timm, then:
        #   encoder = timm.create_model('hrnet_w32', pretrained=True,
        #                               features_only=True, in_chans=1)
        # For DINOv2 (self-supervised ViT, stronger MRI transfer):
        #   dino = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14')
        #   self.encoder = dino  (adapt patch embedding for 1-channel)
        # For MedSAM (ViT-H, medical pretrained):
        #   from segment_anything import sam_model_registry
        #   self.encoder = sam_model_registry['vit_h'](checkpoint='medsam_vit_h.pth')

        if backbone.startswith('hrnet'):
            # HRNet maintains high-res throughout — use timm
            try:
                import timm
                self.encoder = timm.create_model(
                    backbone, pretrained=True, features_only=True, in_chans=1
                )
                # HRNet-w32 feature channels at stride 4: [64, 128, 256, 512]
                # Use the stride-4 branch (highest spatial res) for heatmaps
                self.enc_channels = 64  # stride-4 channel count
                self._hrnet = True
            except ImportError:
                # Fallback to ResNet-50 if timm not available
                import torchvision.models as tv
                resnet = tv.resnet50(weights='IMAGENET1K_V1')
                self.encoder = nn.Sequential(
                    nn.Conv2d(1, 64, 7, 2, 3, bias=False),
                    resnet.bn1, resnet.relu, resnet.maxpool,
                    resnet.layer1, resnet.layer2, resnet.layer3,
                )
                self.enc_channels = 1024
                self._hrnet = False
        else:
            # ViT-based fallback (see DINOv2 / MedSAM notes above)
            import torchvision.models as tv
            self.encoder = tv.vit_b_16(weights='ViT_B_16_Weights.IMAGENET1K_V1')
            self.encoder.conv_proj = nn.Conv2d(1, 768, 16, 16)
            self.enc_channels = 768
            self._hrnet = False

        # ── Slice-Aware Aggregation ───────────────────────────────────
        self.slice_conv = nn.Conv3d(
            self.enc_channels, self.enc_channels,
            kernel_size=(3, 1, 1), padding=(1, 0, 0)
        )

        # ── Cross-Plane Attention ─────────────────────────────────────
        self.cross_plane_attn = CrossPlaneAttention(self.enc_channels, num_heads=8)

        # ── Decoder (only needed for ResNet/ViT; HRNet's high-res branch
        #    already provides full-resolution features) ───────────────
        if self._hrnet:
            # HRNet: minimal head — just a 1×1 conv to K heatmaps
            self.decoder = nn.Conv2d(self.enc_channels, num_landmarks, 1)
        else:
            # ResNet/ViT: U-Net style upsampling decoder
            self.decoder = nn.Sequential(
                nn.ConvTranspose2d(self.enc_channels, 512, 2, 2),
                nn.BatchNorm2d(512), nn.ReLU(inplace=True),
                nn.ConvTranspose2d(512, 256, 2, 2),
                nn.BatchNorm2d(256), nn.ReLU(inplace=True),
                nn.ConvTranspose2d(256, 128, 2, 2),
                nn.BatchNorm2d(128), nn.ReLU(inplace=True),
                nn.ConvTranspose2d(128, 64, 2, 2),
                nn.BatchNorm2d(64), nn.ReLU(inplace=True),
                nn.Conv2d(64, num_landmarks, 1),
            )

        # ── Per-Protocol Slice Classifiers ────────────────────────────
        # Each protocol needs a specific slice; a single scalar is wrong
        # because TT-TG and Patellar Tilt use *different* axial slices.
        # Output: probability distribution over all slices.
        self.slice_classifiers = nn.ModuleDict({
            proto_id: nn.Sequential(
                nn.AdaptiveAvgPool2d(1),
                nn.Flatten(),
                nn.Linear(self.enc_channels, max_slices),
            )
            for proto_id in self.PROTOCOLS
        })

    # ── forward ───────────────────────────────────────────────────────

    def forward(self, v_sag: torch.Tensor, v_cor: torch.Tensor,
                v_ax: torch.Tensor):
        B = v_sag.shape[0]
        S_s, S_c, S_a = v_sag.shape[1], v_cor.shape[1], v_ax.shape[1]

        # 1. Per-slice encode each plane
        f_sag = self._encode_volume(v_sag, S_s)   # [B, S_s, C, h, w]
        f_cor = self._encode_volume(v_cor, S_c)   # [B, S_c, C, h, w]
        f_ax  = self._encode_volume(v_ax,  S_a)   # [B, S_a, C, h, w]

        # 2. Slice-aggregation (3D conv along slice axis)
        f_sag = self.slice_conv(f_sag.permute(0, 2, 1, 3, 4)).permute(0, 2, 1, 3, 4)
        f_cor = self.slice_conv(f_cor.permute(0, 2, 1, 3, 4)).permute(0, 2, 1, 3, 4)
        f_ax  = self.slice_conv(f_ax.permute(0, 2, 1, 3, 4)).permute(0, 2, 1, 3, 4)

        # 3. Cross-plane attention (fully implemented)
        attn_tokens = self.cross_plane_attn(f_sag, f_cor, f_ax)
        # For simplicity, broadcast attended tokens back to spatial features
        # (in production, use token→spatial gating or feature modulation)

        # 4. Decode heatmaps per protocol
        heatmaps: dict[str, torch.Tensor] = {}
        coords:   dict[str, torch.Tensor] = {}
        for proto_id, cfg in self.PROTOCOLS.items():
            plane_feats = {'sagittal': f_sag, 'coronal': f_cor, 'axial': f_ax}[cfg['plane']]
            # Take the middle slice features for decoding
            mid = plane_feats.shape[1] // 2
            feat_2d = plane_feats[:, mid]                  # [B, C, h, w]
            hmap = self.decoder(feat_2d)                   # [B, total_K, H, W]
            # Select this protocol's landmark slice
            start_k = sum(len(p['landmarks']) for p in list(self.PROTOCOLS.values())[:list(self.PROTOCO
            n_k = len(cfg['landmarks'])
            proto_hmap = hmap[:, start_k:start_k + n_k]    # [B, K_p, H, W]
            heatmaps[proto_id] = proto_hmap

            # Soft-argmax → sub-pixel coordinates
            c = self._soft_argmax(proto_hmap)              # [B, K_p, 2]
            coords[proto_id] = c

        # 5. Per-protocol slice classification
        slice_logits: dict[str, torch.Tensor] = {}
        for proto_id, classifier in self.slice_classifiers.items():
            plane_feats = {'sagittal': f_sag, 'coronal': f_cor, 'axial': f_ax}[self.PROTOCOLS[proto_id
            # Mean-pool along slice axis → classify which slice is correct
            pooled = plane_feats.mean(dim=1)               # [B, C, h, w]
            logits = classifier(pooled)                     # [B, max_slices]
            slice_logits[proto_id] = logits

        return {
            'heatmaps':    heatmaps,
            'coords':      coords,
            'slice_logits': slice_logits,
            'features':    {'sagittal': f_sag, 'coronal': f_cor, 'axial': f_ax},
        }

    # ── helpers ───────────────────────────────────────────────────────

    def _encode_volume(self, vol: torch.Tensor, num_slices: int):
        """Encode each slice independently with shared 2D backbone."""
        B = vol.shape[0]
        vol_flat = vol.view(B * num_slices, 1, vol.shape[3], vol.shape[4])
        if self._hrnet:
            # HRNet returns list of feature maps at multiple resolutions
            feats_list = self.encoder(vol_flat)  # list of [B*S, C_i, h_i, w_i]
            feats = feats_list[0]                 # take highest-res (stride 4)
        else:
            feats = self.encoder(vol_flat)        # [B*S, C, h, w]
        _, C, h, w = feats.shape
        return feats.view(B, num_slices, C, h, w)

    @staticmethod
    def _soft_argmax(heatmap: torch.Tensor) -> torch.Tensor:
        """Differentiable argmax via softmax-weighted spatial expectation."""
        B, K, H, W = heatmap.shape
        hmap_flat = heatmap.view(B, K, -1)
        softmax = F.softmax(hmap_flat, dim=-1).view(B, K, H, W)
        yy = torch.arange(H, device=heatmap.device, dtype=torch.float)
        xx = torch.arange(W, device=heatmap.device, dtype=torch.float)
        gy, gx = torch.meshgrid(yy, xx, indexing='ij')
        coord_y = (softmax * gy).sum(dim=(2, 3))
        coord_x = (softmax * gx).sum(dim=(2, 3))
        return torch.stack([coord_x, coord_y], dim=-1)   # [B, K, 2]
```

Architecture Option B: Direct ViT Regression (with DINOv2)

For a simpler, fully-attentional approach. **Use DINOv2** instead of supervised ImageNet ViT — DINOv2's self-supervised patch-level features transfer significantly better to MRI than ImageNet-supervised weights:

```python
class KneeViTRegressor(nn.Module):
    """
    DINOv2-based ViT that takes 2D projections of all 3 planes,
    concatenates them as a multi-channel input, and regresses
    clinical values directly.
    """
    def __init__(self, num_outputs=6):
        super().__init__()
        # DINOv2 transfers better to MRI than supervised ViT
        self.dino = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14')
        # Replace patch embedding to accept 3-channel input (3 planes)
        old_conv = self.dino.patch_embed.proj
        self.dino.patch_embed.proj = nn.Conv2d(
            3, old_conv.out_channels,
            kernel_size=old_conv.kernel_size,
            stride=old_conv.stride
        )
        self.head = nn.Sequential(
            nn.Linear(768, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, num_outputs),
        )

    def forward(self, v_sag, v_cor, v_ax):
        mid_s = v_sag[:, v_sag.shape[1]//2]   # [B, 1, H, W]
        mid_c = v_cor[:, v_cor.shape[1]//2]
        mid_a = v_ax[:,  v_ax.shape[1]//2]
        x = torch.cat([mid_s, mid_c, mid_a], dim=1)  # [B, 3, H, W]
        features = self.dino(x)                       # [B, 768]
        return self.head(features)                    # [B, 6]
```

When to use: Quick baseline. Loses slice localization and cross-plane spatial reasoning. Good for feasibility testing but not for clinical deployment.

Architecture Option C: 3D CNN with Attention

For datasets with high-quality thin-slice (dz < 1 mm) acquisitions:

```python
class Knee3DCNN(nn.Module):
    """
    Full 3D CNN with multi-planar reconstruction awareness.
    Uses a 3D ResNet to process each volume, then fuses with
    cross-volume attention.
    """
    def __init__(self):
        super().__init__()
        # 3D ResNet-18 backbone (per-volume)
        from monai.networks.nets import ResNet
        self.encoder_3d = ResNet(
            block='basic', layers=[2,2,2,2],
            block_inplanes=[64,128,256,512],
            spatial_dims=3, n_input_channels=1
        )
        # Multi-volume fusion transformer
        self.fusion = nn.TransformerEncoderLayer(d_model=512, nhead=8)
        # Heatmap decoder (3D → 2D via MIP or learned projection)
        self.heatmap_head = nn.Sequential(
            nn.Conv3d(512, 256, 1),
            nn.Conv2d(256, 15, 1),  # 15 landmarks
        )
```

When to use: When slice spacing dz ≤ 1.5 mm and you have >500 annotated studies. Otherwise, the 2.5D approach generalizes better.

Data Pipeline Sketch

```
DICOM folders               Annotation CSVs
     │                            │
     ▼                            ▼
┌───────────┐             ┌──────────────────┐
│ DicomLoader │           │ parseSessionCsv/ │
│ (existing)  │           │ parseProtocolCsv │
│ → DicomVolume│          │ → landmarks,     │
│   per plane │           │   measurements,  │
└─────┬─────┘            │   pixel spacing  │
      │                   └────────┬─────────┘
      ▼                            ▼
┌──────────────────────────────────────┐
│          PyTorch Dataset              │
│  __getitem__(patient_id):             │
│    1. Load 3 DicomVolume → Uint8 tensors │
│    2. Normalize to [0,1] or [-1,1]    │
│    3. Resize to fixed spatial dims    │
│       (e.g. 256×256 with aspect-preserving│
│        padding or affine resize)      │
│    4. Load ground-truth landmarks from CSV│
│       Convert CSS px → image px → heatmaps│
│    5. Return (V_S, V_C, V_A), heatmaps, │
│              slice_indices, measurements │
└──────────────────────────────────────┘
```

Data Preprocessing Details

1. **Intensity normalization (Z-score, per-volume):** Knee MRI has significant scanner-to-scanner intensity variation. Use per-volume Z-score normalization instead of percentile clipping: compute μ, σ over *non-zero* voxels only (background = 0), then `volume = (volume - mu) / (sigma + 1e-6)`. This is more robust across vendors (Siemens vs. GE vs. Philips) than min-max normalization.

   ```python
   mask = volume > 0
   mu = volume[mask].mean()
   sigma = volume[mask].std()
   volume = (volume - mu) / (sigma + 1e-6)
   ```

2. **Spatial resampling:** Resize all slices to 256×256 using bilinear interpolation. Do NOT change aspect ratio — pad with zeros to square. Landmark coordinates must be rescaled accordingly.
3. **Slice selection for ViT approach:** Take the middle slice of each volume. For landmark approach: all slices needed.
4. **Heatmap generation:** For each landmark at (x, y) on slice s, generate a 2D Gaussian heatmap with $\sigma = 3$ pixels: $H_{ij} = \exp\left(-\frac{(i-x)^2+(j-y)^2}{2\sigma^2}\right)$
5. **Data augmentation (with elastic deformation):**
   - Random rotation (±10°), random translation (±5%), random brightness (±10%)
   - Random horizontal flip (only if laterality is handled — flip left↔right and swap landmarks)
   - **Random elastic deformation** — the single most effective augmentation for medical landmark detection. Simulates patient positioning variation and soft-tissue deformation. Use `torchio`: `tio.RandomElasticDeformation(num_control_points=7, max_displacement=5)`
   - All spatial transforms must update landmark coordinates accordingly.
6. **Test-Time Augmentation (TTA):** At inference, average predictions over 4 augmented copies (original, flip-LR, ±5° rotation). This reduces landmark error by 10–15% with no additional training cost:

   ```python
   def tta_inference(model, v_sag, v_cor, v_ax):
       preds = []
       for flip in [False, True]:
           for angle in [0, -5, 5]:
               vs, vc, va = apply_transform(v_sag, v_cor, v_ax, flip, angle)
               with torch.no_grad():
                   pred = model(vs, vc, va)
               preds.append(inverse_transform(pred, flip, angle))
       return {k: torch.stack([p[k] for p in preds]).mean(0) for k in preds[0]}
   ```

Loss Functions

```python
class CompositeLoss(nn.Module):
    """
    Multi-task loss: heatmap MSE + Wing loss (coordinates) +
    clinical value L1 + per-protocol cross-entropy (slice classification).

    Wing Loss outperforms L1/MSE for landmark regression because it gives
    higher gradient weight to small errors (critical for sub-mm precision)
    while being robust to outliers via linear tail.
    """
    def __init__(self, lambda_heatmap=1.0, lambda_coord=10.0,
                 lambda_value=0.5, lambda_slice=0.3,
                 label_smoothing=0.1):
        super().__init__()
        self.lambda_heatmap = lambda_heatmap
        self.lambda_coord = lambda_coord
        self.lambda_value = lambda_value
        self.lambda_slice = lambda_slice
        self.mse = nn.MSELoss()
        self.l1 = nn.L1Loss()
        self.wing = WingLoss(w=10.0, epsilon=2.0)
        self.ce = nn.CrossEntropyLoss(label_smoothing=label_smoothing)

    def forward(self, pred, target):
        loss = 0.0
        log = {}

        # 1. Heatmap loss (MSE between predicted & target heatmaps)
        if 'heatmaps' in pred and 'heatmaps' in target:
            l_hm = sum(self.mse(pred['heatmaps'][k], target['heatmaps'][k])
                       for k in pred['heatmaps'])
            loss += self.lambda_heatmap * l_hm
            log['heatmap'] = l_hm.item()

        # 2. Coordinate loss — Wing Loss (higher gradient for small errors)
        if 'coords' in pred and 'coords' in target:
            l_coord = sum(self.wing(pred['coords'][k], target['coords'][k])
                          for k in pred['coords'])
            loss += self.lambda_coord * l_coord
            log['coord'] = l_coord.item()

        # 3. Clinical value loss (L1 on TT-TG, angles, ratios)
        if 'measurements' in pred and 'measurements' in target:
            l_val = sum(self.l1(pred['measurements'][k], target['measurements'][k])
                        for k in pred['measurements'])
            loss += self.lambda_value * l_val
            log['value'] = l_val.item()

        # 4. Per-protocol slice classification (cross-entropy with label smoothing)
        if 'slice_logits' in pred and 'slice_labels' in target:
            l_slice = sum(self.ce(pred['slice_logits'][k], target['slice_labels'][k])
                          for k in pred['slice_logits'])
            loss += self.lambda_slice * l_slice
            log['slice'] = l_slice.item()

        return loss, log
```

Key improvements over the original:

- **Wing Loss** replaces L1 for coordinate regression — gives higher gradient weight to small errors (sub-mm precision matters clinically) while being robust to gross outliers via its linear tail.
- **Per-protocol slice cross-entropy** replaces the buggy scalar MSE on `sigmoid() * S_a`. Each protocol gets its own softmax distribution over slices, with label smoothing (0.1) to prevent overconfidence.
- **Label smoothing** in cross-entropy is critical for clinical deployment — it calibrates confidence scores, which is required for FDA 510(k) submissions.

Training Strategy

Framework: MONAI

Use **MONAI** (Medical Open Network for AI) as the training framework. It handles DICOM loading, 3D augmentations, sliding-window inference, and has production-ready implementations of all architecture options. This replaces most custom preprocessing code:

```
pip install monai[all] torchio timm
```

Pretraining: DINOv2 (not SimCLR/BYOL)

As of 2024, **DINOv2** has become the dominant self-supervised backbone for medical imaging. Use released `dinov2_vitb14` weights directly — they transfer better than ImageNet-supervised weights for MRI:

```python
import torch
dino = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14')
# Freeze DINOv2 backbone, train only the heatmap head for Phase 1
```

Curriculum Learning (Protocol Difficulty Ordering)

Train on protocols with unambiguous landmarks first, then introduce harder ones:

```python
# Phase 1 (epochs 0–30):  Only Patellar Tilt + Sulcus Angle
#   — These have the clearest bony landmarks (condyles, patella)
# Phase 2 (epochs 30–60): Add TT-TG + Insall-Salvati
#   — TT-TG requires perpendicular branch logic; IS needs tendon identification
# Phase 3 (epochs 60–100): Add Caton-Deschamps + Sulcus Angle 3cm
#   — Hardest: CDI requires articular cartilage boundary; SA3cm is cross-plane

protocol_weights = {
    'patellar_tilt':    1.0,   # easiest — clear bony landmarks
    'sulcus_angle':     1.0,   # easiest
```

```python
    'tt_tg':             0.5,   # medium — perpendicular branch
    'insall_salvati':    0.5,   # medium — tendon soft tissue
    'caton_deschamps':   0.3,   # hard — articular surface boundary
    'sulcus_angle_3cm':  0.3,   # hardest — cross-plane inference
}

# Phase schedule: linearly ramp up weights for harder protocols
for epoch in range(num_epochs):
    ramp = min(1.0, epoch / 60) if epoch < 60 else 1.0
    active_weights = {
        k: v * ramp if v < 1.0 else v for k, v in protocol_weights.items()
    }
```

Full Training Loop

```python
# Phase 1: DINOv2 frozen, train heatmap head only
#   ~30 epochs, LR=1e-3, AdamW, cosine schedule

# Phase 2: Unfreeze encoder, full fine-tuning with curriculum
#   ~70 epochs, LR=1e-4, AdamW, cosine schedule with warmup

# Phase 3: Add clinical value loss, fine-tune end-to-end
#   ~50 epochs, LR=5e-5

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
    optimizer, T_0=20, T_mult=2
)

# Use mixed precision for memory efficiency
scaler = torch.cuda.amp.GradScaler()

for epoch in range(num_epochs):
    for batch in dataloader:
        with torch.cuda.amp.autocast():
            pred = model(batch['v_sag'], batch['v_cor'], batch['v_ax'])
            loss, loss_log = criterion(pred, batch['targets'])
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        scheduler.step()
```

Evaluation Metrics

Metric	Formula	Target
Landmark Error (mm)	$\frac{1}{K}\sum_k \lvert\hat{p}_k - p_k\rvert \cdot s$ where s is pixel spacing	< 2 mm (inter-rater variability)
TT-TG MAE	$\lvert\widehat{\text{TT-TG}} - \text{TT-TG}\rvert$	< 2 mm
Angle MAE	$\lvert\hat{\theta} - \theta\rvert$ for Patellar Tilt, Sulcus Angle	< 3°
Ratio MAE	$\lvert\hat{r} - r\rvert$ for Insall–Salvati, Caton–Deschamps	< 0.10
Slice Accuracy	$\%$ of cases where predicted slice = ground-truth slice	> 90%
Clinical Agreement	$\%$ of predictions in same clinical category (normal/borderline/abnormal)	> 85%
Hausdorff Distance (mm)	$\max\left(\sup_{p\in P}\inf_{g\in G}\lvert p-g\rvert, \sup_{g\in G}\inf_{p\in P}\lvert g-p\rvert\right)$ between predicted and ground-truth landmark sets	< 5 mm
Expected Calibration Error (ECE)	$\sum_{m=1}^{M}\frac{	B_m

Why these additions matter:

- **Hausdorff Distance** catches cases where a secondary heatmap mode is placed far from the true landmark — MSE on heatmaps doesn't penalize this heavily, but it produces clinically wrong measurements.
- **ECE (Expected Calibration Error)** measures whether the model's stated confidence matches its empirical accuracy. A well-calibrated model that says "I'm 90% confident this landmark is here" is actually correct 90% of the time. This is **required for FDA 510(k) submissions** and is conspicuously missing from most medical AI evaluation.

  ```python
  from scipy.spatial.distance import directed_hausdorff
  def hausdorff(pred_landmarks, gt_landmarks):
      return max(directed_hausdorff(pred_landmarks, gt_landmarks)[0],
                 directed_hausdorff(gt_landmarks, pred_landmarks)[0])

  def expected_calibration_error(confs, accs, n_bins=10):
      bins = np.linspace(0, 1, n_bins + 1)
      ece = 0.0
      for i in range(n_bins):
          mask = (confs >= bins[i]) & (confs < bins[i+1])
          if mask.sum() > 0:
              ece += (mask.sum() / len(confs)) * abs(accs[mask].mean() - confs[mask].mean())
      return ece
  ```

Inter-rater baseline: Knee MRI measurements have reported inter-rater ICC of 0.80–0.95 depending on protocol. The model should target performance within the clinician inter-rater range.

Dataset Requirements

For training a clinically useful model:

Phase	Minimum	Recommended
Feasibility (ViT regression)	100 annotated studies	300
Landmark heatmap model	300 annotated studies	500+
Clinical validation	50 held-out studies	100 held-out
External validation	30 studies from different scanner	50+

Each "study" = one patient knee (3 DICOM series) with all 6 protocol measurements annotated.

Clinical Gaps & Production Considerations

Annotation Versioning

The CSV exports currently have `annotatorEmail` and `timestamp`, but no `annotationVersion` or `correctionOf` field. Clinicians will revise annotations — you need a way to track which version was used to train each model. **Add these columns to both CSV exports:**

Column	Type	Description
`annotationVersion`	integer	Monotonically increasing; 1 = initial, 2+ = revision
`correctionOf`	UUID or null	If this is a correction, points to the previous `annotationId`
`modelVersion`	string or null	Which model version (if any) generated a suggested annotation that the clinician then corrected

This enables: (a) excluding superseded annotations from training, (b) measuring annotation drift over time, (c) active-learning loops where model predictions are clinician-corrected and fed back.

Two-Stage Inference for Sulcus Angle 3 cm

The Sulcus Angle 3 cm protocol is **cross-plane** and requires two-stage inference — the current architecture sketch doesn't accommodate this properly:

```
Stage 1 (Sagittal):
  Input:  V_S (full sagittal volume)
  Output: Joint line point J = (x_j, y_j) on sagittal slice s_sag
  Compute: Axial slice index s* using DICOM affine:
    patient_3d = sag_affine.voxelToPatient([y_j, x_j, s_sag])
    s* = axial_affine.patientToVoxel(patient_3d).slice + 30mm_offset

Stage 2 (Axial, at slice s*):
  Input:  V_A[:, s*, :, :] (single axial slice at s*)
  Output: Medial condyle M, sulcus S, lateral condyle L
  Compute: Sulcus angle from M, S, L
```

Implementation note: The DICOM affine transforms (`dicomAffine.ts`) already implement `voxelToPatient` and `patientToVoxel`. Port these to PyTorch as differentiable operations so the two-stage pipeline can be trained end-to-end (the slice selection is a hard argmax — use straight-through Gumbel-Softmax or REINFORCE for gradient estimation).

MONAI Integration

Replace custom preprocessing with MONAI's production pipeline:

```python
from monai.transforms import (
    Compose, LoadImageD, ScaleIntensityD, SpatialPadD,
    RandRotateD, RandZoomD, RandGaussianNoiseD,
)
from monai.data import Dataset, DataLoader

transforms = Compose([
    LoadImageD(keys=['v_sag', 'v_cor', 'v_ax']),
    ScaleIntensityD(keys=['v_sag', 'v_cor', 'v_ax']),  # Z-score per-volume
    SpatialPadD(keys=['v_sag', 'v_cor', 'v_ax'], spatial_size=(256, 256)),
    RandRotateD(keys=['v_sag', 'v_cor', 'v_ax'], range_x=0.17, prob=0.5),  # ±10°
    RandZoomD(keys=['v_sag', 'v_cor', 'v_ax'], min_zoom=0.9, max_zoom=1.1, prob=0.5),
])
```

Next Steps

1. **Add `annotationVersion` / `correctionOf` / `modelVersion` columns** to both CSV exports in the UI
2. **Export training data** using the existing CSV export buttons
3. **Validate landmark consistency** — have 2+ clinicians annotate the same 20 studies, compute inter-rater ICC and Hausdorff distance between annotators (this becomes your target performance ceiling)
4. **Set up MONAI + PyTorch training environment** (replaces custom preprocessing)
5. **Train the DINOv2 ViT baseline** (Option B) first — quick to implement, establishes a performance floor
6. **Train the HRNet 2.5D landmark model** (Recommended) with Wing Loss, cross-plane attention, and curriculum learning
7. **Implement two-stage inference** for Sulcus Angle 3 cm (sagittal → affine → axial pipeline)
8. **Calibration study** — measure ECE, apply temperature scaling if needed
9. **Clinical validation study** comparing model vs. clinician measurements on held-out data

---

License

PolyForm Noncommercial License 1.0.0 — free for personal, educational, academic research, and noncommercial clinical use. Commercial use requires a separate license. See LICENSE for full terms.